Free Form Testing Agent

Document: 00_system_overview.md

Version: 2.0
Status: Living Document
Purpose: Source of Truth for Current Architecture
Last Updated: July 2026

---

1. Introduction

This document describes the **current implementation architecture** of the Free Form Testing Agent (FFTA).

Unlike the design documents, this file reflects the **actual implemented system** and should always stay synchronized with the codebase.

Whenever a milestone is completed, this document should be updated.

It serves as the primary reference for understanding:

- What components exist
- Why they exist
- How they interact
- What data flows through them
- What responsibilities each component owns
- Current implementation status
- Future planned components

---

2. Current Project Goal

The current objective of the project is:

> Build an autonomous deterministic exploration engine capable of exploring desktop applications, persisting exploration knowledge, recovering from interruptions, and preparing the foundation for AI-guided testing.

Current application under test:

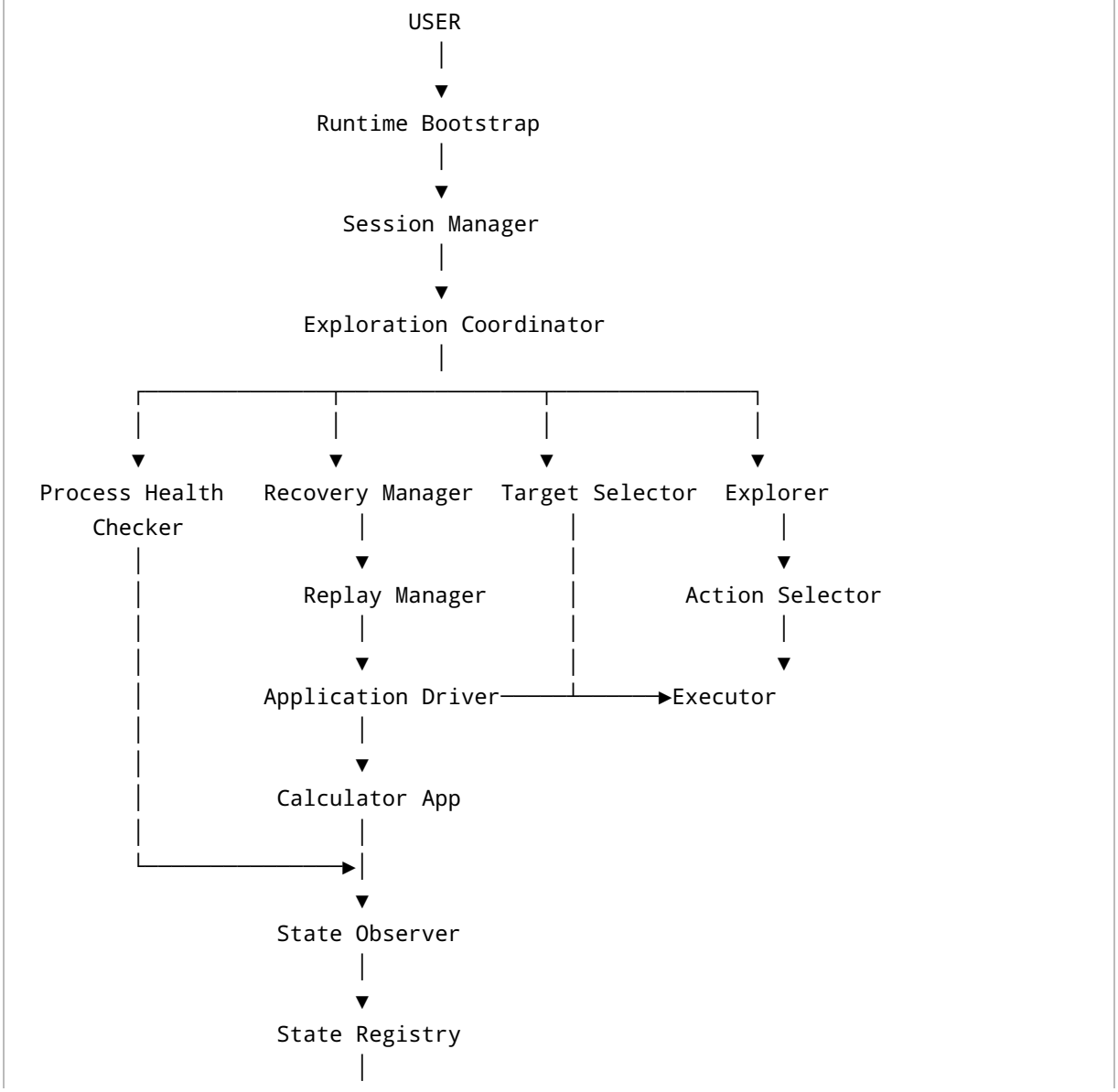
```
Windows Calculator
```

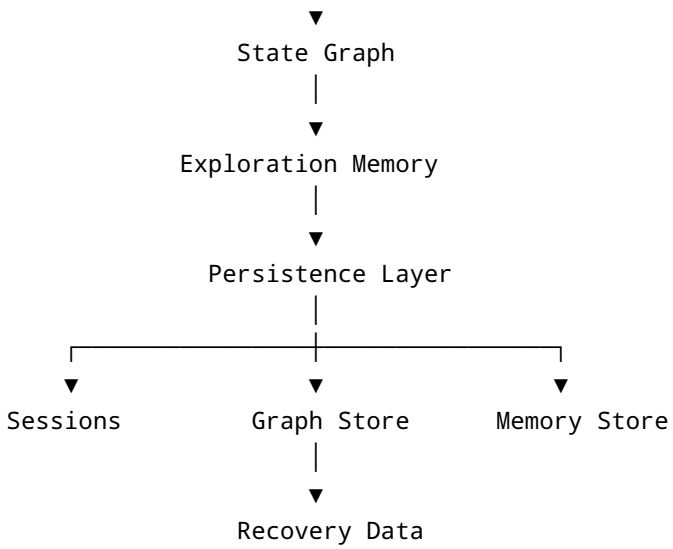
Future targets:

- Windows Desktop Applications
- Web Applications
- Mobile Applications
- APIs

---

3. Current Architecture





---

4. Current Milestone Status

Milestone	Status
Application Driver	✅ Complete
State Observation	✅ Complete
State Model	✅ Complete
State Hashing	✅ Complete
State Graph	✅ Complete
Transition Recording	✅ Complete
Action Selection	✅ Complete
Single Exploration Step	✅ Complete
Exploration Coordinator	✅ Complete
Target Selection	✅ Complete
Replay / State Restoration	✅ Complete
Persistence	✅ Complete
Session Management	✅ Complete
Interrupted Session Recovery	✅ Complete
Process Health Check	✅ Complete

Milestone	Status
Checkpointing	✅ Complete
Resume Exploration	✅ Complete
Failure Detection	🚧 Next
Validation Architecture	⏳ Planned
Requirement Model	⏳ Planned
AI Integration	⏳ Planned

---

5. Runtime Layers

The system is divided into five logical layers.

```
Layer 1
Application Layer

Layer 2
Observation Layer

Layer 3
Exploration Layer

Layer 4
Persistence Layer

Layer 5
Planning Layer (Future AI)
```

---

6. Layer Responsibilities

Layer 1 — Application Layer

Responsible for interacting with the real application.

Components:

- Application Driver
- Calculator Driver

- Executor

Responsibilities:

- Launch application
- Execute actions
- Read controls
- Capture screenshots

Layer 2 — Observation Layer

Responsible for understanding the application.

Components:

- State Observer
- State Registry
- State Signature Generator

Responsibilities:

- Observe UI
- Normalize state
- Remove duplicates
- Generate state identity

Layer 3 — Exploration Layer

Responsible for autonomous exploration.

Components:

- Coordinator
- Explorer
- Target Selector
- Action Selector
- Replay Manager
- Process Health Checker

Responsibilities:

- Explore graph
- Select targets
- Execute actions
- Restore states

- Continue exploration

Layer 4 — Persistence Layer

Responsible for durable storage.

Components:

- Session Repository
- State Repository
- Transition Repository
- Memory Repository

Responsibilities:

- Save progress
- Resume sessions
- Recover runtime

Layer 5 — Planning Layer (Future)

Components:

- Planner
- Requirement Engine
- AI Strategy
- Validator

Responsibilities:

- Understand goals
- Prioritize testing
- Analyze failures
- Guide exploration

---

7. Component Catalog

Runtime Bootstrap

Purpose

Creates every runtime object.

Responsibilities

- Load configuration
- Create repositories
- Create graph
- Create explorer
- Create coordinator
- Start or recover session

Owns

Nothing.

Creates everything.

Session Manager

Purpose

Tracks exploration sessions.

Responsibilities

- Create session
- Load session
- Mark running
- Mark completed
- Mark interrupted

Produces

```
ExplorationSession
```

Exploration Coordinator

Purpose

The central orchestrator.

Responsibilities

- Keep exploration running
- Ask Target Selector

- Ask Replay Manager
- Ask Explorer
- Persist progress
- Stop safely

Does NOT

- Execute actions
- Observe UI
- Generate hashes
- Detect failures

Process Health Checker

Purpose

Ensures the application is healthy before exploration continues.

Responsibilities

- Process exists
- Main window exists
- Window responds
- UI automation works
- Detect restart requirement

Used By

Coordinator

Replay

Recovery

Difference

Health Checker asks:

```
Can exploration continue?
```

Failure Detector asks:

```
Did the application fail?
```

Recovery Manager

Purpose

Rebuild runtime after interruption.

Responsibilities

- Detect interrupted sessions
- Load repositories
- Restore graph
- Restore memory
- Return recovery object

Produces

Recovered runtime.

Replay Manager

Purpose

Restore physical application state.

Responsibilities

- Find replay path
- Restart application
- Execute replay
- Verify states

Depends On

- Graph
- Driver
- Observer

Explorer

Purpose

Perform one complete exploration transaction.

Responsibilities

- Observe state
- Select action
- Execute action
- Observe result
- Register transition
- Update memory

Produces:

```
ExplorationStepResult
```

Target Selector

Purpose

Choose the next state to explore.

Input

Graph

Memory

Coverage

Output

ExplorationTarget

Action Selector

Purpose

Choose the next action.

Input

Current State

Available Actions

Memory

Output

Action

Executor

Purpose

Execute generic actions.

Example

```
CLICK:num7Button
```

becomes

```
Real UI Click
```

Application Driver

Purpose

Communicate with the application.

Current implementation:

Calculator Driver

Future:

Desktop Driver

Web Driver

Mobile Driver

State Observer

Purpose

Convert application UI into structured state.

Produces:

```
ApplicationState
```

State Registry

Purpose

Prevent duplicate states.

Produces:

Existing State

or

New State

Graph Manager

Purpose

Maintain discovered application behavior.

Stores:

- Nodes
- Transitions
- Paths
- Coverage

Exploration Memory

Purpose

Track exploration progress.

Stores:

- Explored actions
- Pending actions
- Visited states
- Current path

Persistence Layer

Purpose

Save runtime state.

Repositories:

- Sessions
- States
- Transitions
- Memory
- Screenshots

---

8. Runtime Data Objects

The system revolves around these core objects.

```
ApplicationState

Action

Transition

StateGraph

ExplorationMemory

ExplorationTarget
```

```
ExplorationSession

ExplorationStepResult

ReplayPath
```

Everything else either creates, modifies, or consumes these objects.

---

9. Runtime Sequence

A normal exploration cycle is:

```
Coordinator
      │
      ▼
Health Check
      │
      ▼
Target Selector
      │
      ▼
Replay Manager
      │
      ▼
Explorer
      │
      ▼
Observer
      │
      ▼
State Registry
      │
      ▼
Action Selector
      │
      ▼
Executor
      │
      ▼
Driver
      │
      ▼
```

```
Application
      │
      ▼
Observer
      │
      ▼
Graph
      │
      ▼
Memory
      │
      ▼
Persistence
      │
      ▼
Coordinator
```

---

10. Recovery Sequence

```
Application Starts
        │
        ▼
Load Last Session
        │
        ▼
Was Session Running?
        │
        ├── No
        │      │
        │      ▼
        │  Create New Session
        │
        ▼
Recovery Manager
        │
        ▼
Load Graph
        │
        ▼
Load Memory
        │
        ▼
Rebuild Runtime
```

```
        |
        ▼
Coordinator
        |
        ▼
Continue Exploration
```

---

11. Data Ownership

Data	Owner
Application UI	Application
Application State	Observer
State Identity	State Registry
Graph	Graph Manager
Explored Actions	Memory
Sessions	Session Manager
Recovery Data	Recovery Manager
Persistence	Repository Layer
Replay Paths	Graph
Runtime Loop	Coordinator

A component should only modify the data it owns.

---

12. Design Principles

The project follows these principles.

Single Responsibility

Every component owns one job.

Deterministic Exploration

Given identical input, exploration produces identical results.

Separation of Concerns

Observation

Execution

Persistence

Planning

remain independent.

Replay Before Branching

The application must always be restored before exploring another branch.

Persistence First

Knowledge is always persisted before advanced AI reasoning.

AI Is Strategic

AI recommends.

The deterministic engine executes.

---

13. Current Architecture Maturity

Current maturity:

```
Core Exploration Engine
≈ 70% Complete
```

Completed:

- Exploration
- Replay

- Recovery
- Persistence
- Coordinator
- Health Monitoring

Remaining:

- Failure Detection
- Validation
- Requirements
- AI Planning
- AI-guided Testing

---

14. Long-Term Vision

The deterministic engine being built today becomes the execution core for the future autonomous testing agent.

```
Requirements
       │
       ▼
AI Planner
       │
       ▼
Coordinator
       │
       ▼
Explorer
       │
       ▼
Application
       │
       ▼
Knowledge Graph
       │
       ▼
AI Analysis
       │
       ▼
Generated Tests
```

The deterministic engine remains the **source of truth**.

AI augments the system with planning, prioritization, reasoning, and interpretation but never replaces the execution infrastructure.

---

15. One-Sentence Summary of Every Component

```
Bootstrap
→ Builds the runtime.

Session Manager
→ Tracks testing sessions.

Coordinator
→ Controls exploration.

Process Health Checker
→ Ensures the application is healthy.

Recovery Manager
→ Rebuilds runtime after interruption.

Replay Manager
→ Restores application state.

Explorer
→ Executes one exploration transaction.

Target Selector
→ Chooses where to explore.

Action Selector
→ Chooses what to do.

Executor
→ Performs the selected action.

Application Driver
→ Talks to the application.

State Observer
→ Understands the application.

State Registry
→ Prevents duplicate states.
```

```
Graph Manager
→ Stores application behavior.

Exploration Memory
→ Tracks exploration progress.

Persistence Layer
→ Saves knowledge.

Failure Detector (Next)
→ Detects deterministic failures.

Validator (Future)
→ Determines correctness.

Planner (Future)
→ Decides testing goals.

AI Strategy (Future)
→ Improves exploration decisions.
```

---

16. Final Mental Model

If you remember only one thing, remember this pipeline:

```
Application
      ↓
Observe
      ↓
Understand
      ↓
Remember
      ↓
Choose Target
      ↓
Restore State
      ↓
Choose Action
      ↓
Execute
      ↓
Observe Again
      ↓
```

```
Update Graph
      ↓
Save Progress
      ↓
Repeat
```

Everything else in the project exists to make this loop reliable, deterministic, recoverable, and eventually intelligent.